PYTHON / HUGGING FACE

기준 모델과 평가 사례

11월 9~15일 · 자유주제 한국어 로컬 챗봇

터미널과 Gradio 웹 화면은 구현 완료 상태다. 2026년 11월 9일 시작을 기준으로 2주간 검수와 모델 비교를 진행하고, 필요한 경우 최대 4주까지 연장한다.

11/09 월

app.py와 web_app.py를 각각 실행한다. Python과 패키지 버전, 실제 모델 장치, 모델 ID, 시스템 메시지, 기록 한도, 생성 토큰 한도를 기록한다.
완료 기준 · 현재 버전의 설정과 실행 방법을 다시 확인할 수 있다.

11/10 화

일상 대화 5개와 사실·개념 설명 5개를 작성한다. 각 사례에 필수 답변 요소를 적고 사실 질문은 확인 근거를 붙인다.
완료 기준 · 답변을 보기 전에 통과 조건이 정해진 사례 10개가 있다.

11/11 수

요약·형식 지시 5개와 후속 대화 5개를 작성한다. 후속 대화는 첫 질문과 이어지는 질문을 순서대로 저장한다.
완료 기준 · 평가 사례 20개와 채점 조건이 준비된다.

11/12 목

대표 사례 3개를 예비 실행한다. 로딩과 생성 시간의 측정 구간을 나누고 워밍업과 메모리 관찰 방법을 정한다.
완료 기준 · 다른 날에도 같은 절차로 측정할 수 있다.

11/13 금

현재 Qwen 모델로 전체 20개를 실행한다. 사례마다 새 대화로 시작하고 후속 질문 세트 안에서는 기록을 유지한다. 실제 답변, 시간, 통과 여부를 저장한다.
완료 기준 · 기준 모델의 원본 응답과 측정값을 확보했다.

11/14 토

실패를 사실 오류, 지시 누락, 맥락 누락, 답변 잘림으로 분류한다. 애매한 채점은 통과 조건과 실제 답변을 다시 대조한다.
완료 기준 · 유형별 통과 수와 대표 실패 예시가 있다.

11/15 일

반복되거나 사용에 영향이 큰 오류 1~2개를 고른다. 후보 모델에 사용할 RAM 또는 VRAM과 다운로드 공간을 확인한다.
완료 기준 · 개선 우선순위와 후보 선정 조건을 기록했다.

PYTHON / HUGGING FACE

설정 개선과 모델 비교

11월 16~22일 · 2주 기본 버전 마감

11/16 월

응답 잘림이 많으면 max_new_tokens, 형식 누락이 많으면 시스템 메시지부터 조정한다. 변경은 한 번에 하나만 적용하고 실패 사례와 기존 통과 사례를 재실행한다.
완료 기준 · 유지하거나 되돌릴 설정을 근거와 함께 정했다.

11/17 화

로컬에서 실행 가능한 Instruct 후보 1개를 선정한다. 모델 카드의 실행 요구사항, 배포 라이선스, 템플릿을 확인한다. 후보 전용 모델·토크나이저를 로딩하고 2~3개 질문으로 예비 실행한다.
완료 기준 · 후보 모델의 ID, 장치, 실행 조건이 기록됐다.

11/18 수

기준 모델, 설정을 조정한 버전, 실행 가능한 후보 모델을 고정 20개 사례로 비교한다. 모델은 하나씩 실행하고 생성 조건 차이를 기록한다.
완료 기준 · 유형별 통과 수, 생성 시간 중앙값, 최대 관측 메모리 비교표가 있다.

11/19 목

한국어 답변, 지시 이행, 맥락 유지 개선 폭과 지연·메모리를 보고 채택할 버전을 고른다. 후보가 자원 한도를 넘으면 기준 모델의 개선 설정을 평가한다.
완료 기준 · 채택한 모델과 선택 이유를 설명할 수 있다.

11/20 금

선택한 모델과 설정을 공통 생성 함수에 적용한다. 두 화면에서 일반 질문, 후속 질문, 빈 입력, 초기화를 검수한다.
완료 기준 · 터미널과 웹에서 같은 모델을 사용하고 주요 기능이 동작한다.

11/21 토

새 가상환경에 requirements.txt로 설치한다. README 절차대로 터미널과 웹을 실행하고 모델 첫 로딩 안내를 확인한다.
완료 기준 · 설치와 실행을 재현했으며 필요한 버전 변경을 반영했다.

11/22 일

성능표, 알려진 한계, 설치 방법을 README와 PROGRESS에 반영한다. 추가로 개선할 오류가 있는지 판단한다.
완료 기준 · 기본 버전이 완료됐고 확장 여부가 기록됐다.

PYTHON / HUGGING FACE

반복 오류 집중 개선

11월 23~29일 · 추가 개선이 필요한 경우

11/23 월
반복 오류 하나를 골라 질문 순서와 기대 답변을 고정한다. 맥락 문제라면 실패 시점에 모델에 전달된 메시지를 확인한다.
완료 기준 · 같은 오류를 재현할 수 있다.

11/24 화
정보가 기록 한도 밖으로 밀렸는지, 입력에 있어도 활용하지 못했는지 구분한다. 비교할 설정 하나와 측정 기준을 정한다.
완료 기준 · 원인 가설과 실험 조건을 기록했다.

11/25 수
기록에서 정보가 빠졌다면 전달 범위를 조정한다. 기록에 남았는데도 틀렸다면 같은 대화를 모델별로 비교한다.
완료 기준 · 변경 전후 답변과 실행 비용이 기록됐다.

11/26 목
이름, 숫자, 조건, 질문 순서를 바꾼 새 사례를 작성해 시험한다.
완료 기준 · 기존 문장에 한정된 개선인지 확인했다.

11/27 금
이전 통과 사례와 초기화, 빈 입력을 두 화면에서 재검수한다. 긴 입력에서 응답 지연도 관찰한다.
완료 기준 · 추가 변경에 따른 회귀와 실행 비용을 확인했다.

11/28 토
원본 응답과 결과표를 대조한다. 측정 오류나 누락이 있는 사례만 같은 조건으로 다시 실행한다.
완료 기준 · 비교에 필요한 결과가 빠짐없이 정리됐다.

11/29 일
통과 수, 새 사례, 지연과 메모리를 보고 변경 채택 여부를 정한다.
완료 기준 · 최종 검수에 사용할 설정이 고정됐다.

PYTHON / HUGGING FACE

최종 검수와 문서화

11월 30일~12월 6일 · 확장 일정 마감

11/30 월

확정 설정으로 고정 사례 20개를 재실행한다. 최초 모델과 최종 버전의 유형별 통과 수와 응답 예시를 비교한다.
완료 기준 · 최초 대비 개선과 남은 오류를 설명할 수 있다.

12/01 화

최종 모델의 로딩 시간, 생성 시간 중앙값, 최대 관측 RAM을 정리한다. GPU를 사용했다면 전용 메모리를 따로 적는다.
완료 기준 · 성능표에 실행 장치와 측정 조건이 포함됐다.

12/02 수

새 가상환경에서 requirements.txt로 설치한다. 문서에 적힌 명령으로 app.py와 web_app.py를 실행한다.
완료 기준 · 두 실행 경로가 새 환경에서 재현된다.

12/03 목

일반 질문, 후속 질문, 빈 입력, 초기화를 확인한다. 재현한 문제의 조치 내용을 PROGRESS에 추가한다.
완료 기준 · 마지막 기능 검수 결과가 기록됐다.

12/04 금

README에 최종 모델 ID, 실행 방법, 대화 예시, 알려진 한계를 반영한다. 설치 버전과 requirements.txt를 대조한다.
완료 기준 · 설치 안내와 실제 배포 설정이 일치한다.

12/05 토

저장소에 포함할 변경을 Sourcetree에서 확인한다. 가상환경, 캐시, 인증 정보가 스테이징되지 않았는지 점검한다.
완료 기준 · 게시할 코드와 문서의 변경 범위가 확인됐다.

12/06 일

코드, 의존성, 계획, 진행 기록, 성능 비교 결과를 점검하고 최종 완료 상태를 기록한다.
완료 기준 · 선택 근거와 재현 절차가 저장소 문서에 남았다.

PYTHON / HUGGING FACE

개발 환경과 의존성

편집기 · 실행 환경 · 패키지 설치

VS Code
는 두 Python 파일을 편집하고 통합 터미널에서 실행하는 데 사용한다. Python: Select Interpreter에서 프로젝트의 .venv를 선택한다. 맥락 오류를 조사할 때 generate_reply() 진입점에 중단점을 걸어 메시지 수와 순서를 확인한다.

Python
은 사용자 입력과 명령을 처리하고, 메시지를 role과 content 필드가 있는 딕셔너리로 만든다. app.py는 터미널 반복 입력과 공통 생성 함수를 담당한다. web_app.py는 웹 입력을 같은 함수에 넘긴다.

venv
는 프로젝트마다 Python 패키지 설치 위치를 분리한다. 기존 환경에서 개발하고, 별도의 새 환경에서 문서대로 설치할 수 있는지 검수한다.

pip
는 가상환경의 python.exe -m pip를 통해 실행한다. requirements.txt의 직접 의존성은 현재 torch 2.15.0, transformers 5.22.2, gradio 6.31.0으로 고정돼 있다. 변경 시 새 환경에서 확인한 버전을 반영한다.

PYTHON / HUGGING FACE

모델 다운로드와 생성

Hugging Face Hub · Transformers · PyTorch

Hugging Face Hub
는 공개 모델 파일과 토크나이저의 배포처다. from_pretrained()에 모델 ID를 넘겨 가중치와 토크나이저를 받고 이후 실행에는 로컬 캐시를 재사용한다. 후보 비교 시 모델 ID와 사용한 실행 조건을 기록한다.

Qwen2.5-0.5B-Instruct
는 최초 비교 기준인 대화용 모델이다. 현재 모델을 고정 질문 20개로 먼저 평가한 뒤, 장치와 메모리에 맞는 후보를 같은 절차로 검증한다.

Transformers의 AutoTokenizer
는 모델에 맞는 텍스트 변환 규칙을 로딩한다. apply_chat_template()은 시스템 메시지, 최근 사용자·챗봇 발화, 새 질문을 모델이 이해하는 토큰 입력으로 바꾼다. add_generation_prompt=True로 챗봇 발화 시작 위치를 표시한다.

Transformers의 AutoModelForCausalLM
은 사전 학습된 모델을 불러온다. generate()에 현재 max_new_tokens=128을 적용하고, 출력에서 입력 토큰 길이만큼 제외한 뒤 새 토큰을 디코딩한다. 두 인터페이스가 이 함수를 공유하므로 설정 변경을 한곳에서 적용할 수 있다.

PyTorch
는 모델 가중치와 입력 텐서를 관리하고 실제 추론 연산을 실행한다. 현재 코드에는 실행 장치 지정이 없으므로 평가 전에 모델의 실제 장치를 확인한다. 후보에 GPU를 사용한다면 모델과 입력을 같은 장치에 두고, 비교 결과에 장치 차이를 표시한다. model.eval()을 유지해 평가 모드에서 비교한다.

Qwen 모델 카드
Transformers 채팅 템플릿

PYTHON / HUGGING FACE

화면과 평가 도구

터미널 · Gradio · 측정 · 버전 관리

터미널 채팅
은 app.py에서 input()으로 질문을 받고 공백만 입력하면 건너뛴다. /reset은 기록을 비우고 /exit는 반복을 종료한다. 최근 최대 6개 메시지와 새 질문을 전달한다.

Gradio
는 web_app.py의 ChatInterface로 브라우저 입력과 기록을 받는다. 콜백에서 사용자와 챗봇의 텍스트 메시지를 추려 최근 6개를 공통 함수에 넘긴다. launch(share=False)로 로컬에서 열고 화면에서 기록을 지운 뒤 새 요청을 확인한다.

Python time
의 perf_counter()로 모델 로딩 시간과 답변 생성 시간을 분리한다. 첫 워밍업 응답은 따로 표시한다. 같은 사례들의 생성 시간 중앙값을 비교하고 답변 길이 차이도 함께 확인한다.

Windows 작업 관리자
에서 모델을 하나씩 실행하며 Python 프로세스의 최대 관측 RAM을 기록한다. GPU 사용 시 전용 GPU 메모리는 별도 열에 기록한다.

Git · Sourcetree · GitHub
로 로컬 변경을 검토하고 버전을 관리한다. 기준 모델과 설정 개선을 구분할 수 있게 변경 단위를 나누고, 검수한 코드와 문서를 커밋한다. .gitignore로 가상환경, 캐시, 토큰 파일을 제외한다.

Gradio ChatInterface 소스

완료 기준

두 화면에서 일반 질문, 후속 질문, 빈 입력, 초기화를 검수한다. 새 가상환경에서 설치와 실행을 재현하고, 최종 모델의 선택 근거와 성능 비교 결과를 기록한다.